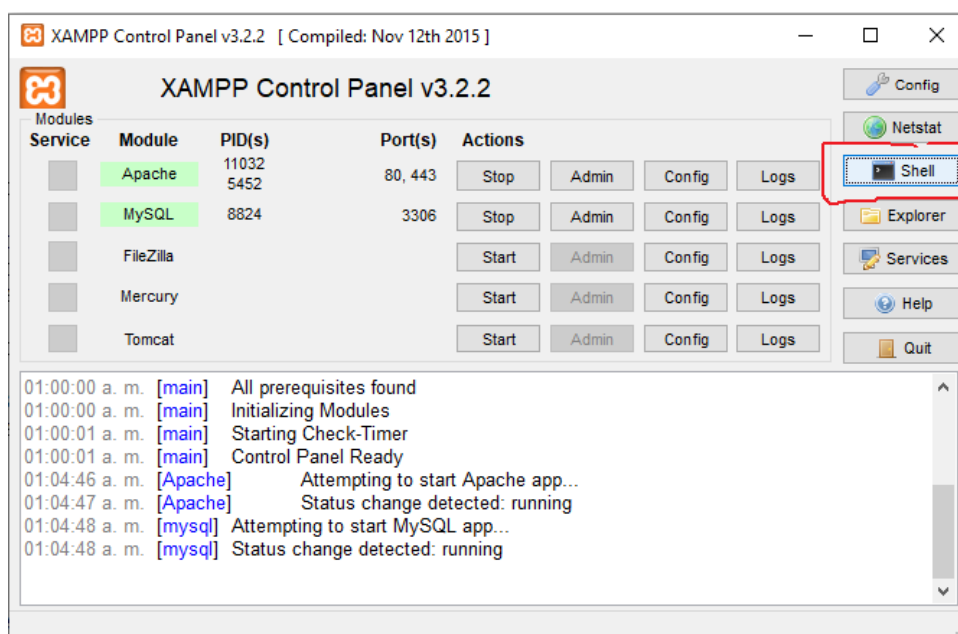


TUTORIAL BÁSICO MYSQL (consola)

Introducción

El objetivo de este tutorial es mostrar el uso del programa cliente mysql para crear y usar una sencilla base de datos. mysql (algunas veces referido como "monitor mysql") es un programa interactivo que permite conectarnos a un servidor MySQL, ejecutar algunas consultas, y ver los resultados. mysql puede ser usado también en modo batch: es decir, se pueden colocar toda una serie de consultas en un archivo, y posteriormente decirle a mysql que ejecute dichas consultas.

Este tutorial asume que mysql está instalado en alguna máquina y que disponemos de un servidor MySQL al cuál podemos conectarnos. Si este no es el caso, tenemos que contactar con nuestro administrador MySQL. (Si nosotros somos los administradores, es necesario consultar la documentación de MySQL que se refiere a la instalación y configuración del servidor MySQL).



Se mostrara una consola con la siguiente información:

```
Setting environment for using XAMPP for windows.
jose1@DESKTOP-ECD7NCE c:\xampp
#
```

A continuación se describe el proceso completo de creación y uso de una base de datos en MySQL. Si se está interesado sólo en el acceso y uso de una base de datos existente, se pueden omitir las secciones que describen como crear la base de datos y las tablas correspondientes.

Puesto que es imposible que se describan a detalle muchos de los tópicos cubiertos en este artículo, se recomienda que se consulte el manual de MySQL para obtener más información al respecto.

Conectándose y desconectándose al servidor MySQL

Para conectarse al servidor, usualmente necesitamos de un nombre de usuario (login) y de una contraseña (password), y si el servidor al que nos deseamos conectar está en una máquina diferente de la nuestra, también necesitamos indicar el nombre o la dirección IP de dicho servidor. Una vez que conocemos estos tres valores, podemos conectarnos de la siguiente manera:

```
# mysql -u NombreDeUsuario -h NombreDelServidor -p
```

Cuando ejecutamos este comando, se nos pedirá que proporcionemos también la contraseña para el nombre de usuario que estamos usando.

Si la conexión al servidor MySQL se pudo establecer de manera satisfactoria, recibiremos el mensaje de bienvenida y estaremos en el prompt de mysql:

```
# mysql -u blueman -h casita -p Enter  
password: *****
```

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
  
Your MariaDB connection id is 325  
  
Server version: 10.1.25-MariaDB mariadb.org binary distribution  
  
Copyright (c) 2000, 2017, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input  
statement.  
  
MariaDB [(none)]>
```

Este prompt nos indica que mysql está listo para recibir comandos.

Algunas instalaciones permiten que los usuarios se conecten de manera anónima al servidor corriendo en la máquina local. Si es el caso de nuestra máquina, debemos de ser capaces de conectarnos al servidor invocando a mysql sin ninguna opción:

```
MariaDB [(none)]>
```

Después de que nos hemos conectado de manera satisfactoria, podemos desconectarnos en cualquier momento al escribir "quit" o "exit".

La mayoría de los ejemplos siguientes asume que estamos conectados al servidor, lo cual se indica con el prompt de mysql.

Ejecutando algunas consultas

En este momento debimos de haber podido conectarnos ya al servidor MySQL, aún cuando no hemos seleccionado alguna base de datos para trabajar. Lo que haremos a continuación es escribir algunos comandos para irnos familiarizando con el funcionamiento de mysql

```
MariaDB [(none)]> SELECT VERSION(), CURRENT_DATE;
```

+	+	+
	VERSION()	
+	+	+
	3.23.41	
+	+	+
	CURRENT_DATE	
+	+	+
	2002-10-01	
+	+	+

1 row in set (0.03 sec)

```
MariaDB [(none)]>
```

Este comando ilustra distintas cosas acerca de mysql:

- Un comando normalmente consiste de un sentencia SQL seguida por un punto y coma.
- Cuando emitimos un comando, mysql lo manda al servidor para que lo ejecute, nos muestra los resultados y regresa el prompt indicando que está listo para recibir más consultas.
- mysql muestra los resultados de la consulta como una tabla (filas y columnas). La primera fila contiene etiquetas para las columnas. Las filas siguientes muestran los resultados de la consulta. Normalmente las etiquetas de las columnas son los nombres de los campos de las tablas que estamos usando en alguna consulta. Si lo que estamos recuperando es el valor de una expresión (como en el ejemplo anterior) las etiquetas en las columnas son la expresión en sí.
- mysql muestra cuántas filas fueron regresadas y cuanto tiempo tardó en ejecutarse la consulta, lo cual puede darnos una idea de la eficiencia del servidor, aunque estos valores pueden ser un tanto imprecisos ya que no se muestra la hora del CPU, y porque pueden verse afectados por otros factores, tales como la carga del servidor y la velocidad de comunicación en una red.

- Las palabras clave pueden ser escritas usando mayúsculas y minúsculas.

Las siguientes consultas son equivalentes:

```
MariaDB [(none)]> SELECT VERSION(), CURRENT_DATE;
MariaDB [(none)]> select
version(), current_date;
MariaDB [(none)]> SeLeCt
vErSiOn(), current_DATE;
Aquí está otra consulta que demuestra como se pueden escribir
algunas expresiones matemáticas y trigonométricas:
MariaDB [(none)]> SELECT SIN(PI()/4), (4+1)*5;
```

SIN(PI()/4)	(4+1)*5
0.707107	25

Aunque hasta este momento se han escrito sentencias sencillas de una sólo línea, es posible escribir más de una sentencia por línea, siempre y cuando estén separadas por punto y coma:

```
MariaDB [(none)]> SELECT VERSION(); SELECT NOW();
```

VERSION()
3.23.41

1 row in set (0.01 sec)

NOW()
2002-10-28 14:26:04

1 row in set (0.01 sec)

Un comando no necesita ser escrito en una sólo línea, así que los comandos que requieran de varias líneas no son un problema. mysql determinará en dónde finaliza la sentencia cuando encuentre el punto y coma, no cuando encuentre el fin de línea.

Aquí está un ejemplo que muestra un consulta simple escrita en varias líneas:

```
MariaDB [(none)]> SELECT
-> USER(),
-> CURRENT_DATE;
```

USER()	CURRENT_DATE
blueman@localhost	2002-09-14

1 row in set (0.00 sec)

```
MariaDB [(none)]>
```

En este ejemplo debe notarse como cambia el prompt (de MariaDB [(none)]> a ->) cuando se escribe una consulta en varias líneas. Esta es la manera en cómo mysql indica que está esperando a que finalice la consulta.

De nuevo, se nos regresa el comando el prompt MariaDB [(none)]> que nos indica que mysql está listo para una nueva consulta.

En la siguiente tabla se muestran cada uno de los prompts que podemos obtener y una breve descripción de su significado para mysql:

Prompt	Significado
MariaDB [(none)]>	Listo para una nueva consulta.
->	Esperando la línea siguiente de una consulta multi-línea.
'>	Esperando la siguiente línea para completar una cadena que comienza con una comilla sencilla (').
">	Esperando la siguiente línea para completar una cadena que comienza con una comilla doble (").

Los comandos multi-línea comúnmente ocurren por accidente cuando tecleamos ENTER, pero olvidamos escribir el punto y coma. En este caso mysql se queda esperando para que finalicemos la consulta:

```
MariaDB [(none)]> SELECT USER()  
->
```

Si esto llega a suceder, muy probablemente mysql estará esperando por un punto y coma, de manera que si escribimos el punto y coma podremos completar la consulta y mysql podrá ejecutarla:

```
MariaDB [(none)]> SELECT USER()  
-> ;  
+-----+  
| USER() |  
+-----+  
| root@localhost |  
+-----+  
1 row in set (0.00 sec)
```

```
MariaDB [(none)]>
```

Los prompts '>' y '>' ocurren durante la escritura de cadenas. En mysql podemos escribir cadenas utilizando comillas sencillas o comillas dobles (por ejemplo, 'hola' y "hola"), y mysql nos permite escribir cadenas que ocupen múltiples líneas. De manera que cuando veamos el prompt '>' o '>', mysql nos indica que hemos empezado a escribir una cadena, pero no la hemos finalizado con la comilla correspondiente.

Aunque esto puede suceder si estamos escribiendo una cadena muy grande, es más frecuente que obtengamos alguno de estos prompts si inadvertidamente escribimos alguna de estas comillas.

Por ejemplo:

```
MariaDB [(none)]> SELECT * FROM mi_tabla WHERE nombre = "Lupita AND edad < 30;  
>
```

Si escribimos esta consulta SELECT y entonces presionamos ENTER para ver el resultado, no sucederá nada. En lugar de preocuparnos porque la consulta ha tomado mucho tiempo, debemos notar la pista que nos da mysql cambiando el prompt. Esto nos indica que mysql está esperando que finalicemos la cadena iniciada("Lupita).

El prompt cambiará de nuevo al ya conocido MariaDB [(none)]> , indicándonos que mysql está listo para una nueva consulta.

Es sumamente importante conocer lo que significan los prompts '>' y '>', ya que si en algún momento nos aparece alguno de ellos, todas la líneas que escribamos a continuación serán consideradas como parte de la cadena, inclusive cuando escribimos QUIT. Esto puede ser confuso, especialmente si no sabemos que es necesario escribir la comilla correspondiente para finalizar la cadena, para que podamos escribir después algún otro comando, o terminar la consulta que deseamos ejecutar.

Creando y usando una base de datos

Ahora que conocemos como escribir y ejecutar sentencias, es tiempo de acceder a una base de datos.

Supongamos que tenemos diversas mascotas en casa (nuestro pequeño zoológico) y deseamos tener registros de los datos acerca de ellas. Podemos hacer esto al crear tablas que guarden esta información, para que posteriormente la consulta de estos datos sea bastante fácil y de manera

muy práctica. Esta sección muestra como crear una base de datos, crear una tabla, incorporar datos en una tabla, y recuperar datos de las tablas de diversas maneras.

La base de datos "zoológico" será muy simple (deliberadamente), pero no es difícil pensar de situaciones del mundo real en la cual una base de datos similar puede ser usada.

Primeramente usaremos la sentencia SHOW para ver cuáles son las bases de datos existentes en el servidor al que estamos conectados:

```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| mysql    |
| test     |
+-----+
2 rows in set (0.00 sec)

MariaDB [(none)]>
```

Es probable que la lista de bases de datos que veamos sea diferente en nuestro caso, pero seguramente las bases de datos "mysql" y "test" estarán entre ellas. En particular, la base de datos "mysql" es requerida, ya que ésta tiene la información de los privilegios de los usuarios de MySQL. La base de datos "test" es creada durante la instalación de MySQL con el propósito de servir como área de trabajo para los usuarios que inician en el aprendizaje de MySQL.

Se debe anotar también que es posible que no veamos todas las bases de datos si no tenemos el privilegio SHOW DATABASES. Se recomienda revisar la sección del manual de MySQL dedicada a los comandos GRANT y REVOKE.

Si la base de datos "test" existe, hay que intentar acceder a ella:

```
MariaDB [(none)]> USE test
Database changed
MariaDB [(none)]>
```

Observar que USE, al igual que QUIT, no requieren el uso del punto y coma, aunque si se usa éste, no hay ningún problema. El comando USE es especial también de otra manera: éste debe ser usado en una sola línea.

Podríamos usar la base de datos "test" (si tenemos acceso a ella) para los ejemplos que vienen a continuación, pero cualquier cosa que hagamos puede ser eliminada por cualquier otro usuario que tenga acceso a esta base de datos. Por esta razón, es recomendable que preguntemos al administrador MySQL acerca de la base de datos que podemos usar. Supongamos que deseamos tener una base de datos llamada "zoologico" (nótese que no se está acentuando la palabra) a la cual sólo nosotros tengamos acceso, para ello el administrador necesita ejecutar un comando como el siguiente:

```
MariaDB [(none)]> GRANT ALL on zoologico.* TO MiNombreUsuario@MiComputadora
-> IDENTIFIED BY 'MiContraseña';
```


en donde *MiNombreUsuario* es el nombre de usuario asignado dentro del contexto de MySQL, *MiComputadora* es el nombre o la dirección IP de la computadora desde la que nos conectamos al servidor MySQL, y *MiContraseña* es la contraseña que se nos ha asignado, igualmente, dentro del ambiente de MySQL exclusivamente. Ambos, nombre de usuario y contraseña no tienen nada que ver con el nombre de usuario y contraseña manejados por el sistema operativo (si es el caso).

Si el administrador creó la base de datos al momento de asignar los permisos, podemos hacer uso de ella. De otro modo, nosotros debemos crearla:

```
MariaDB [(none)]> USE zoologico
ERROR 1049: Unknown database 'zoologico'
MariaDB [(none)]>
```

El mensaje anterior indica que la base de datos no ha sido creada, por lo tanto necesitamos crearla.

```
MariaDB [(none)]> CREATE DATABASE
zoologico; Query OK, 1 row affected
(0.00 sec)

MariaDB [(none)]> USE zoologico
Database changed
MariaDB [(none)]>
```

Bajo el sistema operativo Unix, los nombres de las bases de datos son sensibles al uso de mayúsculas y minúsculas (no como las palabras clave de SQL), por lo tanto debemos de tener cuidado de escribir correctamente el nombre de la base de datos. Esto es cierto también para los nombres de las tablas.

Al crear una base de datos no se selecciona ésta de manera automática; debemos hacerlo de manera explícita, por ello usamos el comando USE en el ejemplo anterior.

La base de datos se crea sólo una vez, pero nosotros debemos seleccionarla cada vez que iniciamos una sesión con mysql. Por ello es recomendable que se indique la base de datos sobre la que vamos a trabajar al momento de invocar al monitor de MySQL. Por ejemplo:

```
# mysql -h casita -u bluman -p zoologico

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 17 to server version: 3.23.38-nt

Type 'help;' or '\h' for help. Type '\c' to clear the buffer

MariaDB [(none)]>
```

Observar que "zoologico" no es la contraseña que se está proporcionando desde la línea de comandos, sino el nombre de la base de datos a la que deseamos acceder. Si deseamos proporcionar la contraseña en la línea de comandos después de la opción "-p", debemos de hacerlo sin dejar espacios (por ejemplo, **-phola123**, no como **-p hola123**). Sin embargo, escribir nuestra contraseña desde la línea de comandos no es recomendado, ya que es bastante inseguro.

Creando una tabla

Crear la base de datos es la parte más fácil, pero en este momento la base de datos está vacía, como lo indica el comando SHOW TABLES:

```
MariaDB [(none)]> SHOW TABLES;  
Empty set (0.00 sec)
```

La parte un tanto complicada es decidir la estructura que debe tener nuestra base de datos: qué tablas se necesitan y qué columnas estarán en cada tabla.

En principio, necesitamos una tabla que contenga un registro para cada una de nuestras mascotas. Ésta puede ser una tabla llamada mascotas, y debe contener por lo menos el nombre de cada uno de nuestros animalitos. Ya que el nombre en sí no es muy interesante, la tabla debe contener alguna otra información. Por ejemplo, si más de una persona en nuestra familia tiene una mascota, es probable que tengamos que guardar la información acerca de quien es el dueño de cada mascota. Así mismo, también sería interesante contar con alguna información más descriptiva tal como la especie, y el sexo de cada mascota.

¿Y que sucede con la edad?. Esto puede ser también de interés, pero no es una buena idea almacenar este dato en la base de datos. La edad cambia conforme pasa el tiempo, lo cual significa que debemos de actualizar los registros frecuentemente. En vez de esto, es una mejor idea guardar un valor fijo, tal como la fecha de nacimiento. Entonces, cuando necesitemos la edad, la podemos calcular como la diferencia entre la fecha actual y la fecha de nacimiento. MySQL proporciona funciones para hacer operaciones entre fechas, así que no hay ningún problema.

Al almacenar la fecha de nacimiento en lugar de la edad tenemos algunas otras ventajas:

Podemos usar la base de datos para tareas tales como generar recordatorios para cada cumpleaños próximo de nuestras mascotas. Podemos calcular la edad en relación a otras fechas que la fecha actual. Por ejemplo, si almacenamos la fecha en que murió nuestra mascota en la base de datos, es fácil calcular que edad tenía nuestro animalito cuando falleció. Es probable que estemos pensando en otro tipo de información que sería igualmente útil en la tabla "mascotas", pero para nosotros será suficiente por ahora contar con información de nombre, propietario, especie, nacimiento y fallecimiento.

Usaremos la sentencia CREATE TABLE para indicar como estarán conformados los registros de nuestras mascotas.

```
MariaDB [(none)]> CREATE TABLE mascotas(  
  -> nombre VARCHAR(20), propietario VARCHAR(20),  
  -> especie VARCHAR(20), sexo VARCHAR(1), nacimiento DATE,  
  -> fallecimiento DATE);  
Query OK, 0 rows affected (0.02 sec)  
  
MariaDB [(none)]>
```

VARCHAR es una buena elección para los campos nombre, propietario, y especie, ya que los valores que almacenarán son de longitud variable. No es necesario que la longitud de estas columnas sea la misma, ni tampoco que sea de 20. Se puede especificar cualquier longitud entre 1 y 255, lo que se considere más adecuado. Si resulta que la elección de la longitud de los campos que hemos hecho no resultó adecuada, MySQL proporciona una sentencia ALTER TABLE que nos puede ayudar a solventar este problema.

El campo sexo puede ser representado en una variedad de formas, por ejemplo, "m" y "f", o tal vez "masculino" y "femenino", aunque resulta más simple la primera opción.

El uso del tipo de dato DATE para los campos nacimiento y fallecimiento debe de resultar obvio.

Ahora que hemos creado la tabla, la sentencia SHOW TABLES debe producir algo como:

```
MariaDB [(none)]> SHOW TABLES;
```

```
+-----+
| Tables_in_zoologico |
+-----+
| mascotas            |
+-----+
1 row in set (0.00 sec)
```

```
MariaDB [(none)]>
```

Para verificar que la tabla fué creada como nosotros esperabamos, usaremos la sentencia DESCRIBE:

```
MariaDB [(none)]> DESCRIBE mascotas;
```

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null  | Key  | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| nombre     | varchar(20) | YES   |      | NULL    |       |
| propietario | varchar(20) | YES   |      | NULL    |       |
| especie    | varchar(20) | YES   |      | NULL    |       |
| sexo       | char(1)     | YES   |      | NULL    |       |
| nacimiento | date       | YES   |      | NULL    |       |
| fallecimiento | date      | YES   |      | NULL    |       |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

```
MariaDB [(none)]>
```

Podemos hacer uso de la sentencia DESCRIBE en cualquier momento, por ejemplo, si olvidamos los nombres ó el tipo de las columnas en la tabla.

Cargando datos en una tabla

Después de haber creado la tabla, ahora podemos incorporar algunos datos en ella, para lo cual haremos uso de las sentencias INSERT y LOAD DATA.

Supongamos que los registros de nuestras mascotas pueden ser descritos por los datos mostrados en la siguiente tabla.

Nombre	Propietario	Especie	Sexo	Nacimiento	Fallecimiento
Fluffy	Arnoldo	Gato	f	1999-02-04	
Mau	Juan	Gato	m	1998-03-17	
Buffy	Arnoldo	Perro	f	1999-05-13	
FanFan	Benito	Perro	m	2000-08-27	
Kaiser	Diana	Perro	m	1998-08-31	1997-07-29
Chispa	Omar	Ave	f	1998-09-11	
Wicho	Tomás	Ave		2000-02-09	
Skim	Benito	Serpiente	m	2001-04-29	

Debemos observar que MySQL espera recibir fechas en el formato YYYY-MM-DD, que puede ser diferente a lo que nosotros estamos acostumbrados.

Ya que estamos iniciando con una tabla vacía, la manera más fácil de poblarla es crear un archivo de texto que contenga un registro por línea para cada uno de nuestros animalitos para que posteriormente carguemos el contenido del archivo en la tabla únicamente con una sentencia.

Por tanto, debemos de crear un archivo de texto "mascotas.txt" que contenga un registro por línea con valores separados por tabuladores, cuidando que el orden de las columnas sea el mismo que utilizamos en la sentencia CREATE TABLE. Para valores que no conozcamos podemos usar valores nulos (NULL). Para representar estos valores en nuestro archivo debemos usar \N.

El archivo mascotas.txt contendrá:

```
Fluffy Arnoldo Gato f 1999-02-04 \N
Mau Juan Gato m 1998-03-17 \N
Buffy Arnoldo Perro f 1999-05-13 \N
FanFan Benito Perro m 2000-08-27 \N
Kaiser Diana Perro m 1998-08-31 1997-07-29
Chispa Omar Ave f 1998-09-11 \N
Wicho Tomás Ave \N 2000-02-09 \N
Skim Benito Serpiente m 2001-04-29 \N
```

Para cargar el contenido del archivo en la tabla mascotas, usaremos el siguiente comando:

```
MariaDB [(none)]> LOAD DATA LOCAL INFILE "mascotas.txt" INTO TABLE mascotas;
```

La sentencia LOAD DATA nos permite especificar cuál es el separador de columnas, y el separador de registros, por default el tabulador es el separador de columnas (campos), y el salto de línea es el separador de registros, que en este caso son suficientes para que la sentencia LOAD DATA lea correctamente el archivo "mascotas.txt".

Si lo que deseamos es añadir un registro a la vez, entonces debemos hacer uso de la sentencia INSERT. En la manera más simple, debemos proporcionar un valor para cada columna en el orden en el cual fueron listados en la sentencia CREATE TABLE. Supongamos que nuestra hermana Diana compra un nuevo hamster nombrado Pelusa. Podemos usar la sentencia INSERT para agregar su registro en nuestra base de datos.

```
MariaDB [(none)]> INSERT INTO `mascotas` (`nombre`, `propietario`, `especie`,
`sexo`, `nacimiento`, `fallecimiento`) VALUES
('Pelusa','Diana','Hamster','f','2000-03-30',NULL);
```

Notar que los valores de cadenas y fechas deben estar encerrados entre comillas. También, con la sentencia INSERT podemos insertar el valor NULL directamente para representar un valor nulo, un valor que no conocemos. En este caso no se usa \N como en el caso de la sentencia LOAD DATA.

De este ejemplo, debemos ser capaces de ver que es un poco más la tarea que se tiene que realizar si inicialmente cargamos los registros con varias sentencias INSERT en lugar de una única sentencia LOAD DATA.

Recuperando información de una tabla

La sentencia SELECT es usada para obtener la información guardada en una tabla. La forma general de esta sentencia es:

```
SELECT LaInformaciónQueDeseamos FROM DeQueTabla WHERE CondiciónASatisfacer
```

Aquí, *LaInformaciónQueDeseamos* es la información que queremos ver. Esta puede ser una lista de columnas, o un * para indicar "todas las columnas". *DeQueTabla* indica el nombre de la tabla de la cual vamos a obtener los datos. La cláusula WHERE es opcional. Si está presente, la *CondiciónASatisfacer* especifica las condiciones que los registros deben satisfacer para que puedan ser mostrados.

- Seleccionando todos los datos

La manera más simple de la sentencia SELECT es cuando se recuperan todos los datos de una tabla:

```
MariaDB [(none)]> SELECT * FROM mascotas;
```

nombre	propietario	especie	sexo	nacimiento	fallecimiento
Fluffy	Arnoldo	Gato	f	1999-02-04	NULL
Mau	Juan	Gato	m	1998-03-17	NULL
Buffy	Arnoldo	Perro	f	1999-05-13	NULL
FanFan	Benito	Perro	m	2000-08-27	NULL
Kaiser	Diana	Perro	m	1998-08-31	1997-07-29
Chispa	Omar	Ave	f	1998-09-11	NULL
Wicho	Tomás	Ave	NULL	2000-02-09	NULL
Skim	Benito	Serpiente	m	2001-04-29	NULL
Pelusa	Diana	Hamster	f	2000-03-30	NULL

9 rows in set (0.00 sec)

Esta forma del SELECT es útil si deseamos ver los datos completos de la tabla, por ejemplo, para asegurarnos de que están todos los registros después de la carga de un archivo.

Por ejemplo, en este caso que estamos tratando, al consultar los registros de la tabla, nos damos cuenta de que hay un error en el archivo de datos (mascotas.txt): parece que *Kaiser* ha nacido después de que ha fallecido!. Al revisar un poco el pedigree de *Kaiser* encontramos que la fecha correcta de nacimiento es el año 1989, no 1998.

Hay por lo menos un par de maneras de solucionar este problema:

Editar el archivo "mascotas.txt" para corregir el error, eliminar los datos de la tabla mascotas con la sentencia DELETE, y cargar los datos nuevamente con el comando LOAD DATA:

```
MariaDB [(none)]> DELETE FROM mascotas;  
MariaDB [(none)]> LOAD DATA LOCAL INFILE "mascotas.txt" INTO TABLE mascotas;
```

Sin embargo, si hacemos esto, debemos ingresar los datos de *Pelusa*, la mascota de nuestra hermana Diana.

La segunda opción consiste en corregir sólo el registro erróneo con una sentencia UPDATE:

```
MariaDB [(none)]> UPDATE mascotas SET nacimiento="1989-08-31" WHERE
```

Como se mostró anteriormente, es muy fácil recuperar los datos de una tabla completa. Pero típicamente no deseamos hacer esto, particularmente cuando las tablas son demasiado grandes. En vez de ello, estaremos más interesados en responder preguntas particulares, en cuyo caso debemos especificar algunas restricciones para la información que deseamos ver.

- Seleccionando registros particulares

Podemos seleccionar sólo registros particulares de una tabla. Por ejemplo, si deseamos verificar el cambio que hicimos a la fecha de nacimiento de *Kaiser*, seleccionamos sólo el registro de *Kaiser* de la siguiente manera:

```
MariaDB [(none)]> SELECT * FROM mascotas WHERE nombre="Kaiser";  
+-----+-----+-----+-----+-----+-----+  
| nombre | propietario | especie | sexo | nacimiento | fallecimiento |  
+-----+-----+-----+-----+-----+-----+  
| Kaiser | Diana      | Perro   | m    | 1989-08-31 | 1997-07-29   |  
+-----+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

La salida mostrada confirma que el año ha sido corregido de 1998 a 1989.

La comparación de cadenas es normalmente no sensitiva, así que podemos especificar el nombre como "kaiser", "KAISER", etc. El resultado de la consulta será el mismo.

Podemos además especificar condiciones sobre cualquier columna, no sólo el "nombre". Por ejemplo, si deseamos conocer qué mascotas nacieron después del 2000, tendríamos que usar la

columna "nacimiento":

```
MariaDB [(none)]> SELECT * FROM mascotas WHERE nacimiento >= "2000-1-1";
```

nombre	propietario	especie	sexo	nacimiento	fallecimiento
FanFan	Benito	Perro	m	2000-08-27	NULL
Wicho	Tomás	Ave	NULL	2000-02-09	NULL
Skim	Benito	Serpiente	m	2001-04-29	NULL
Pelusa	Diana	Hamster	f	2000-03-30	NULL

4 rows in set (0.00 sec)

Podemos también combinar condiciones, por ejemplo, para localizar a los perros hembras:

```
MariaDB [(none)]> SELECT * FROM mascotas WHERE especie="Perro" AND sexo="f";
```

nombre	propietario	especie	sexo	nacimiento	fallecimiento
Buffy	Arnoldo	Perro	f	1999-05-13	NULL

1 row in set (0.00 sec)

La consulta anterior usa el operador lógico AND. Hay también un operador lógico OR:

```
MariaDB [(none)]> SELECT * FROM mascotas WHERE especie = "Ave" OR especie = "Gato";
```

nombre	propietario	especie	sexo	nacimiento	fallecimiento
Fluffy	Arnoldo				
Mau	Juan	Gato	f	1999-02-04	NULL
		Gato	m	1998-03-17	NULL
Chispa	Omar	Ave	f	1998-09-11	NULL
Wicho	Tomás	Ave	NULL	2000-02-09	NULL

4 rows in set (0.00 sec)

El operador AND y el operador OR pueden ser intercambiados. Si hacemos esto, es buena idea usar paréntesis para indicar como deben ser agrupadas las condiciones:

```
MariaDB [(none)]> SELECT * FROM mascotas WHERE (especie = "Gato" AND sexo = "m")  
-> OR (especie = "Perro" AND sexo = "f");
```

nombre	propietario	especie	sexo	nacimiento	fallecimiento
Mau	Juan	Gato	m	1998-03-17	NULL
Buffy	Arnoldo	Perro	f	1999-05-13	NULL

2 rows in set (0.00 sec)

- Seleccionando columnas particulares

Si no deseamos ver los registros completos de una tabla, entonces tenemos que usar los nombres de las columnas en las que estamos interesados separándolas por coma. Por ejemplo, si deseamos conocer la fecha de nacimiento de nuestras mascotas, debemos seleccionar la columna "nombre" y "nacimiento":

```
MariaDB [(none)]> SELECT nombre, nacimiento FROM mascotas;
```

```
+-----+-----+
| nombre | nacimiento |
+-----+-----+
| Fluffy | 1999-02-04 |
| Mau    | 1998-03-17 |
| Buffy  | 1999-05-13 |
| FanFan | 2000-08-27 |
| Kaiser | 1989-08-31 |
| Chispa | 1998-09-11 |
| Wicho  | 2000-02-09 |
| Skim   | 2001-04-29 |
| Pelusa | 2000-03-30 |
+-----+-----+
9 rows in set (0.00 sec)
```

Para conocer quién tiene alguna mascota, usaremos la siguiente consulta:

```
MariaDB [(none)]> SELECT propietario FROM mascotas;
```

```
+-----+
| propietario |
+-----+
| Arnoldo     |
| Juan        |
| Arnoldo     |
| Benito      |
| Diana       |
| Omar        |
| Tomás       |
| Benito      |
| Diana       |
+-----+
9 rows in set (0.00 sec)
```

Sin embargo, debemos notar que la consulta recupera el nombre del propietario de cada mascota, y algunos de ellos aparecen más de una vez. Para minimizar la salida, agregaremos la palabra clave DISTINCT:

```
MariaDB [(none)]> SELECT DISTINCT propietario FROM mascotas;
```

```
+-----+
| propietario |
+-----+
| Arnoldo     |
| Juan        |
+-----+
```



```
| Benito |
| Diana |
| Omar  |
| Tomás |
+-----+
6 rows in set (0.03 sec)
```

Se puede usar también una cláusula WHERE para combinar selección de filas con selección de columnas. Por ejemplo, para obtener la fecha de nacimiento de los perritos y los gatitos, usaremos la siguiente consulta:

```
MariaDB [(none)]> SELECT nombre, especie, nacimiento FROM mascotas
-> WHERE especie = "perro" OR especie = "gato";
```

nombre	especie	nacimiento
Fluffy	Gato	1999-02-04
Mau	Gato	1998-03-17
Buffy	Perro	1999-05-13
FanFan	Perro	2000-08-27
Kaiser	Perro	1989-08-31

```
+-----+
5 rows in set (0.00 sec)
```

Ordenando registros

Se debe notar en los ejemplos anteriores que las filas regresadas son mostradas sin ningún orden en particular. Sin embargo, frecuentemente es más fácil examinar la salida de una consulta cuando las filas son ordenadas en alguna forma útil. Para ordenar los resultados, tenemos que usar una cláusula ORDER BY.

Aquí aparecen algunos datos ordenados por fecha de nacimiento:

```
MariaDB [(none)]> SELECT nombre, nacimiento FROM mascotas ORDER BY nacimiento;
```

nombre	nacimiento
Kaiser	1989-08-31
Mau	1998-03-17
Chispa	1998-09-11
Fluffy	1999-02-04
Buffy	1999-05-13
wicho	2000-02-09
Pelusa	2000-03-30
FanFan	2000-08-27
Skim	2001-04-29

```
+-----+
9 rows in set (0.00 sec)
```

En las columnas de tipo caracter, el ordenamiento es ejecutado normalmente de forma no sensitiva, es decir, no hay diferencia entre mayúsculas y minúsculas. Sin embargo, se puede forzar un ordenamiento sensitivo al usar el operador BINARY.

Para ordenar en orden inverso, debemos agregar la palabra clave DESC al nombre de la columna que estamos usando en el ordenamiento:

```
MariaDB [(none)]>SELECT nombre, nacimiento FROM mascotas ORDER BY
-> nacimiento DESC;
+-----+-----+
| nombre | nacimiento |
+-----+-----+
| Skim   | 2001-04-29 |
| FanFan | 2000-08-27 |
| Pelusa | 2000-03-30 |
| Wicho  | 2000-02-09 |
| Buffy  | 1999-05-13 |
| Fluffy | 1999-02-04 |
| Chispa | 1998-09-11 |
| Mau    | 1998-03-17 |
| Kaiser | 1989-08-31 |
+-----+-----+
9 rows in set (0.00 sec)
```

Podemos ordenar múltiples columnas. Por ejemplo, para ordenar por tipo de animal, y poner al inicio los animalitos más pequeños de edad, usaremos la siguiente consulta:

```
MariaDB [(none)]> SELECT nombre, especie, nacimiento FROM mascotas
-> ORDER BY especie, nacimiento DESC;
+-----+-----+-----+
| nombre | especie | nacimiento |
+-----+-----+-----+
| wicho  | Ave     | 2000-02-09 |
| Chispa | Ave     | 1998-09-11 |
| Fluffy | Gato    | 1999-02-04 |
| Mau    | Gato    | 1998-03-17 |
| Pelusa | Hamster | 2000-03-30 |
| FanFan | Perro   | 2000-08-27 |
| Buffy  | Perro   | 1999-05-13 |
| Kaiser | Perro   | 1989-08-31 |
| Skim   | Serpiente | 2001-04-29 |
+-----+-----+-----+
9 rows in set (0.00 sec)
```

Notar que la palabra clave DESC aplica sólo a la columna nombrada que le precede.